

C++ Multithreading Interview Prep Kit (Pure pthreads Edition)

Contents

C++ Multithreading Interview Prep Kit (Pure pthreads Edition)	1
GLOSSARY (reference throughout)	1
TIER 0 — MENTAL MODEL (read once, don't skip)	3
TIER 1 — BASICS: THREADS, MUTEXES, ARGUMENT PASSING	4
TIER 2 — PRODUCER-CONSUMER FAMILY (the highest-frequency interview question)	8
TIER 3 — PRIORITY-QUEUE-BASED TIMERS	13
TIER 4 — THREAD POOLS	17
TIER 5 — ADVANCED: ATOMICS, MEMORY ORDERING, LOCK-FREE	21
TIER 6 — GOTCHAS, EDGE CASES, AND THINGS THAT TRIP PEOPLE UP LIVE	26
DIFFICULTY-ORDERED QUESTION INDEX (for structured practice)	27
SUGGESTED STUDY / PRACTICE SCHEDULE	28

C++ Multithreading Interview Prep Kit (Pure pthreads Edition)

pthread_create, pthread_mutex_t, pthread_cond_t, producer-consumer, priority timers, thread pools

Scope note: this edition uses **only the raw POSIX pthreads C API** — no `std::thread`, `std::mutex`, `std::condition_variable`, `std::future`, `std::atomic`, or any other C++ standard-library concurrency wrapper. Every function is `pthread_*` or POSIX (`sem_t`, `pthread_rwlock_t`). This is the right kit to drill if an interviewer explicitly wants pthreads/C-style code (common in embedded, kernel, driver, and older systems codebases), or if you just want to be fluent at the level below the C++ wrappers. Every code sample was compiled and run with `gcc -pthread` (files are plain C, `.c`, since that's pthreads' native habitat — C++ code can call the same API identically by including `<pthread.h>`, and I'll note the handful of places C++ makes something cleaner, like using a `struct` with a constructor instead of manual init calls).

This kit mirrors the structure of the `std::` edition tier-for-tier, so you can cross-reference: same glossary, same six tiers, same interview questions — every answer is just reimplemented in raw pthreads. Every question is tagged with a difficulty rating from **[Difficulty 1/5 - Easy]** to **[Difficulty 5/5 - Very Hard]**, and there's a difficulty-ordered index near the end (before the study schedule) if you'd rather drill strictly easiest-to-hardest instead of tier-by-tier. Thread *arguments* are passed via plain structs (the pthreads-native way); where object-oriented state genuinely helps (the thread pool, the future), a C++ class is used with a small **static trampoline function** bridging it to pthreads' required `void* (*)(void*)` callback signature — this pattern is explained in full the first time it appears (4.1). No lambda functions appear anywhere in this kit, including in `demo main()` code.

GLOSSARY (reference throughout)

Term	Meaning
Thread	An independent sequence of execution within a process; shares heap/globals with other threads in the process, has its own stack, registers, program counter.
Mutex (MUTual EXclusion)	A lock that only one thread can hold at a time. Protects a critical section. In pthreads: <code>pthread_mutex_t</code> .
Critical Section	Code that accesses shared state and must not run concurrently on more than one thread.

Term	Meaning
Condition Variable (CV)	A wait/notify mechanism that lets a thread sleep until some condition becomes true, without busy-waiting. Always paired with a mutex. In pthreads: <code>pthread_cond_t</code> .
Spurious Wakeup	A condition variable wait can return even though nobody signaled it. This is why you always re-check the predicate in a <code>while</code> loop, never an <code>if</code> .
Race Condition	Program behavior depends on the non-deterministic timing/interleaving of threads; usually a bug.
Data Race	Specifically: two threads access the same memory location, at least one is a write, with no synchronization ordering between them. Undefined behavior. Race condition is the broader logical term; data race is the precise memory-model term.
Deadlock	Two or more threads are each waiting on a resource the other holds; nobody proceeds.
Livelock	Threads keep changing state in response to each other but no thread makes progress.
Starvation	A thread never gets scheduled/never acquires a lock because others keep getting priority.
Priority Inversion	A low-priority thread holds a lock a high-priority thread needs, and a medium-priority thread preempts the low-priority one, indirectly blocking the high-priority thread.
Atomic Operation	An operation that appears indivisible to other threads. pthreads itself has no atomics API — you'd reach for GCC/Clang <code>__atomic_* / __sync_*</code> builtins, or C11 <code><stdatomic.h></code> , since pure POSIX pthreads predates a standard atomics API.
CAS	Compare-And-Swap — an atomic primitive: “if the value is still X, set it to Y, else tell me it changed.” Building block for lock-free structures.
False Sharing	Two threads write to different variables that happen to live on the same CPU cache line, causing needless cache-coherency traffic and slowdown — a performance bug, not a correctness bug.
Thread Pool	A fixed (or bounded) set of worker threads that pull tasks from a queue, avoiding the cost of spawning/destroying threads per task.
Producer-Consumer	A pattern where “producer” threads generate work/data items and “consumer” threads process them, coordinated through a shared bounded/unbounded queue.
Reentrant Lock / Recursive Mutex	A mutex that the <i>same</i> thread can lock multiple times without deadlocking itself. In pthreads: a <code>pthread_mutex_t</code> initialized with <code>PTHREAD_MUTEX_RECURSIVE</code> via <code>pthread_mutexattr_settype</code> .
Futex	Fast Userspace mutEX — the Linux kernel primitive <code>pthread_mutex_t</code> is built on: uncontended lock/unlock is pure userspace CAS, and it only traps into the kernel when there's actual contention.
Thundering Herd	When <code>pthread_cond_broadcast</code> wakes all waiting threads, but only one can actually proceed — the rest just re-check and go back to sleep, wasting CPU cycles.
Semaphore	A counter-based synchronization primitive; <code>sem_wait</code> blocks if count is 0, otherwise decrements; <code>sem_post</code> increments. POSIX: <code>sem_t</code> .

Term	Meaning
TLS	Thread-Local Storage — each thread gets its own separate copy of a variable. In pthreads: pthread_key_t + pthread_key_create/pthread_setspecific/pthread_getspecific (or the compiler extension __thread/C11 _Thread_local, which is simpler but not part of the pthreads API proper).
ABA Problem	In lock-free CAS code: a value changes from A → B → A between your read and your CAS, so CAS succeeds even though the world changed underneath you.
Detached Thread	A thread whose resources are automatically released when it finishes, since nobody will pthread_join it. Set via pthread_detach.
Joinable Thread	The default state — another thread must call pthread_join to reclaim its resources and retrieve its return value, or it becomes a zombie/leaks resources.

TIER 0 — MENTAL MODEL (read once, don't skip)

Why threads exist: A process gets one address space. Threads let multiple execution contexts share that address space (globals, heap) cheaply, instead of paying for separate address spaces (processes + IPC). The cost of that sharing is: you must explicitly coordinate access to anything mutable and shared, or you get undefined behavior.

The three things you're always juggling in these problems: 1. **Mutual exclusion** — only one thread touches this data at a time (pthread_mutex_t). 2. **Signaling** — a thread needs to *wait* until some condition is true, and be woken efficiently rather than spin (pthread_cond_t). 3. **Lifetime/ownership** — who joins/detaches each thread, and how do you shut down cleanly without leaking threads or use-after-free.

The pattern that answers 80% of these interview questions:

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
// shared state protected by mtx, e.g. a queue

/* Consumer side: */
pthread_mutex_lock(&mtx);
while (!(/* predicate: queue not empty, or done */)) {
    pthread_cond_wait(&cv, &mtx); /* atomically unlocks mtx while sleeping, re-locks
    before returning */
}
/* ... do work with the shared state, still holding mtx ... */
pthread_mutex_unlock(&mtx); /* release before doing slow work, if any */

/* Producer side: */
pthread_mutex_lock(&mtx);
/* push item into shared state */
pthread_mutex_unlock(&mtx);
pthread_cond_signal(&cv); /* or pthread_cond_broadcast */
```

Say this pattern out loud in every interview. It's the thing they're checking you actually understand, not just memorized.

Key API surface you must have memorized cold:

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
```

```

int pthread_detach(pthread_t thread);

int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex); /* returns EBUSY immediately if already
locked */
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);

int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);
int pthread_cond_signal(pthread_cond_t *cond); /* wakes exactly one waiter */
int pthread_cond_broadcast(pthread_cond_t *cond); /* wakes all waiters */
int pthread_cond_destroy(pthread_cond_t *cond);

```

Every function returns 0 on success and a positive `errno`-style value on failure (they do NOT set the global `errno` — they return the error code directly). Real production code checks these returns; interview code often skips the checks for brevity, but say out loud that you’d check them in production, and DO check them on `pthread_create` at minimum since a failed thread creation silently not running your worker is a nasty bug to chase.

All start routines have the fixed signature `void (*)(void *)`. This is the single biggest mechanical annoyance versus `std::thread` (which accepts any callable with any signature) — you must package arguments into a struct and pass `(void*)&args`, and cast back inside the thread function. Every example below shows this pattern.

TIER 1 — BASICS: THREADS, MUTEXES, ARGUMENT PASSING

1.1 [Difficulty 1/5 - Easy] Q: Write a program that spawns N threads, each increments a shared counter 100,000 times, and prints the final value. Explain what happens without a mutex, then fix it.

What they’re testing: do you understand *why* data races happen at the instruction level, can you fix it with a mutex, and can you correctly package/pass thread arguments through `void*`.

Broken version (for discussion, don’t submit as final):

```

#include <pthread.h>
#include <stdio.h>

long counter = 0;

void *increment_broken(void *arg) {
    (void)arg;
    for (int i = 0; i < 100000; ++i) {
        counter++; /* NOT atomic: load, add 1, store back -- 3 steps */
    }
    return NULL;
}

int main() {
    pthread_t threads[4];
    for (int i = 0; i < 4; ++i) {
        pthread_create(&threads[i], NULL, increment_broken, NULL);
    }
    for (int i = 0; i < 4; ++i) {
        pthread_join(threads[i], NULL);
    }
    printf("Final (broken): %ld\n", counter); /* usually less than 400000 -- see note below */
}

```

```

    return 0;
}

```

Explanation to give out loud: counter++ compiles to load-register, add, store-register. If thread A loads 5, thread B loads 5 before A stores, both compute 6 and store 6 — one increment is lost. This is a data race: undefined behavior, not just “wrong answer.”

Practical note worth mentioning to an interviewer: whether you actually *observe* corruption depends on optimization level, iteration count, and core count/scheduling — at -O0 with enough iterations (try 2,000,000 with 8 threads) the corruption is easy to reproduce reliably; at higher optimization levels the compiler may keep the loop counter in a register in a way that happens to reduce (but never eliminates the *legality* of) the race window on a given run. Saying “this is undefined behavior regardless of whether I can get it to misbehave on this machine right now” is the correct and precise thing to say — a race you can’t currently reproduce is still a bug.

Fix — mutex:

```

#include <pthread.h>
#include <stdio.h>

long counter = 0;
pthread_mutex_t counter_mtx = PTHREAD_MUTEX_INITIALIZER;

void *increment_safe(void *arg) {
    (void)arg;
    for (int i = 0; i < 100000; ++i) {
        pthread_mutex_lock(&counter_mtx);
        counter++;
        pthread_mutex_unlock(&counter_mtx);
    }
    return NULL;
}

int main() {
    pthread_t threads[4];
    for (int i = 0; i < 4; ++i) {
        pthread_create(&threads[i], NULL, increment_safe, NULL);
    }
    for (int i = 0; i < 4; ++i) {
        pthread_join(threads[i], NULL);
    }
    printf("Final (safe): %ld\n", counter); /* always 400000 */
    return 0;
}

```

Follow-up they may ask: “Locking/unlocking per iteration is slow — how would you speed this up?” Answer: have each thread accumulate into a private local counter across the whole loop, and only take the lock once at the end to add the local sum into the shared counter. This turns 100,000 lock/unlock pairs into 1 per thread:

```

void *increment_fast(void *arg) {
    (void)arg;
    long local = 0;
    for (int i = 0; i < 100000; ++i) {
        local++;
        /* no lock needed -- purely thread-local */
    }
    pthread_mutex_lock(&counter_mtx);
    counter += local;
    /* one lock/unlock per thread total, not per increment */
    pthread_mutex_unlock(&counter_mtx);
    return NULL;
}

```

This “accumulate locally, merge once” pattern is one of the highest-value things to mention proactively — it shows you think about lock contention as a performance axis, not just a correctness checkbox.

1.2 [Difficulty 1/5 - Easy] Q: How do you pass multiple arguments into a pthread, since pthread_create only accepts a single void*?

Answer: Package the arguments into a struct, pass a pointer to it (heap-allocated if the caller’s stack frame might not outlive the thread — very important gotcha), and cast back inside the thread function.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    int iterations;
} WorkerArgs;

void *worker(void *arg) {
    WorkerArgs *args = (WorkerArgs *)arg;
    for (int i = 0; i < args->iterations; ++i) {
        /* do work using args->id */
    }
    printf("worker %d finished %d iterations\n", args->id, args->iterations);
    free(args); /* IMPORTANT: whoever heap-allocated it must free it -- decide ownership
                  up front */
    return NULL;
}

int main() {
    pthread_t threads[4];
    for (int i = 0; i < 4; ++i) {
        WorkerArgs *args = malloc(sizeof(WorkerArgs)); /* heap alloc: survives after main's
                                                         loop moves on */
        args->id = i;
        args->iterations = 1000;
        pthread_create(&threads[i], NULL, worker, args);
    }
    for (int i = 0; i < 4; ++i) {
        pthread_join(threads[i], NULL);
    }
    return 0;
}
```

The classic bug this question is fishing for: passing a pointer to a stack-local variable that changes or goes out of scope before the thread reads it.

```
/* BUG: passing &i directly -- by the time the thread runs, i may have
   already changed value (or the loop may have exited, if i is scoped
   to the loop and threads run after main() moves on) */
for (int i = 0; i < 4; ++i) {
    pthread_create(&threads[i], NULL, worker_bad, &i); /* WRONG: races on shared i */
}
```

Always either heap-allocate a per-thread copy (shown above) or, if the array of arg structs is guaranteed to outlive all the threads (e.g. a stack array declared before the loop, joined before the function returns), pass a pointer into a *distinct* element of that array for each thread — never the loop variable itself.

1.3 [Difficulty 2/5 - Easy-Medium] Q: What is a deadlock? Give a minimal example and two ways to prevent it.

Answer: Deadlock — thread A holds lock 1 and wants lock 2; thread B holds lock 2 and wants lock 1. Neither proceeds.

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_a(void *arg) {
    (void)arg;
    pthread_mutex_lock(&m1);
    usleep(10000);           /* widen the window to make the race visible */
    pthread_mutex_lock(&m2); /* waits forever if thread_b already holds m2 */
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}

void *thread_b(void *arg) {
    (void)arg;
    pthread_mutex_lock(&m2);
    usleep(10000);
    pthread_mutex_lock(&m1); /* classic deadlock: waits on what thread_a holds */
    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

Prevention 1 — lock ordering (the standard real-world fix): always acquire locks in the same global order everywhere in the codebase (e.g., always m1 before m2, by convention or by comparing mutex addresses/IDs if the set of locks is dynamic). This is the fix you should lead with in a pthreads-only interview, since pthreads has no built-in multi-lock primitive.

Prevention 2 — trylock-and-backoff: attempt the second lock with `pthread_mutex_trylock`; if it fails (returns `EBUSY`), release the first lock and retry from scratch after a backoff, rather than blocking and risking deadlock.

```
void *thread_a_safe(void *arg) {
    (void)arg;
    for (;;) {
        pthread_mutex_lock(&m1);
        if (pthread_mutex_trylock(&m2) == 0) {
            /* got both locks -- critical section */
            pthread_mutex_unlock(&m2);
            pthread_mutex_unlock(&m1);
            break;
        }
        pthread_mutex_unlock(&m1); /* couldn't get m2 -- back off and retry, don't hold m1
                                   and block */
        usleep(1000);             /* small backoff to avoid busy-spinning */
    }
    return NULL;
}
```

Mention both; lock ordering is simpler and is what real codebases actually enforce via convention/lint rules, trylock-backoff is the answer when ordering genuinely can't be established up front (e.g. locking two arbitrary nodes in a graph where there's no natural order).

1.4 [Difficulty 1/5 - Easy] Q: What's a spurious wakeup, and why do you always wait in a while loop, never an if?

Answer: POSIX condition variables are explicitly permitted by the standard to wake a waiting thread even when no one called `pthread_cond_signal/broadcast` — a spurious wakeup, allowed for implementation efficiency reasons. Additionally, even with a real signal, by the time your thread wakes and `pthread_cond_wait` re-acquires the mutex, another thread might have already consumed the resource you were waiting for.

So the correct pattern is always:

```
pthread_mutex_lock(&mtx);
while (!condition_is_true()) {      /* NOT if */
    pthread_cond_wait(&cv, &mtx);
}
/* ... proceed, mutex is held again here ... */
pthread_mutex_unlock(&mtx);
```

This is non-negotiable in raw pthreads — unlike `std::condition_variable`, there is no predicate-overload convenience form; you write the while loop by hand every single time. Interviewers specifically watch for whether you write while or if here.

1.5 [Difficulty 2/5 - Easy-Medium] Q: pthread_mutex_t has no direct “unlock on scope exit” (RAII) like std::lock_guard. How do you avoid leaking a lock on an early return or exception path?

Answer: Raw C has no destructors, so the discipline is manual: **every exit path out of a locked region must explicitly unlock.** In C, that means no early return while holding a lock without unlocking first — often enforced with a single exit point (goto cleanup; pattern) rather than scattering return statements.

```
int do_work_with_lock(SomeState *state) {
    pthread_mutex_lock(&state->mtx);
    int result = -1;

    if (!validate(state)) {
        goto unlock_and_return; /* still unlocks -- don't just `return -1;` here */
    }
    result = compute(state);

unlock_and_return:
    pthread_mutex_unlock(&state->mtx);
    return result;
}
```

If you're allowed to write this in C++ (interviewer says “pthreads API is fine but you can compile as C++”), you get RAII back for free by wrapping the mutex in a tiny guard class — this is worth mentioning as the practical real-world answer even in a “pure pthreads” interview, since production C++ codebases that use pthreads directly (rather than `std::mutex`) almost always do exactly this:

```
class PthreadMutexGuard {
public:
    explicit PthreadMutexGuard(pthread_mutex_t *m) : mtx_(m) { pthread_mutex_lock(mtx_); }
    ~PthreadMutexGuard() { pthread_mutex_unlock(mtx_); }
    PthreadMutexGuard(const PthreadMutexGuard &) = delete;
    PthreadMutexGuard &operator=(const PthreadMutexGuard &) = delete;
private:
    pthread_mutex_t *mtx_;
};
// usage: PthreadMutexGuard lock(&state->mtx); -- unlocks automatically on any return/exception
```

Mention this proactively — a five-line RAII wrapper around `pthread_mutex_t` is a “shows real engineering judgment” move, even in a strictly-C interview where you’d describe it verbally instead of typing it.

TIER 2 — PRODUCER-CONSUMER FAMILY (the highest-frequency interview question)

2.1 [Difficulty 3/5 - Medium] Q: Implement a thread-safe bounded blocking queue (classic producer-consumer). Producers block when full, consumers block when empty.

What they’re testing: this is THE canonical question. They want correct mutex+CV usage, two separate conditions (not-full / not-empty), correct while-loop waiting, and a clean shutdown protocol. Expect 20-30 min for this alone.

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define CAPACITY 10

typedef struct {
    int buffer[CAPACITY];
    int head, tail, count;
    pthread_mutex_t mutex;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
    int shutdown;
} BoundedQueue;

void queue_init(BoundedQueue *q) {
    q->head = q->tail = q->count = 0;
    q->shutdown = 0;
    pthread_mutex_init(&q->mutex, NULL);
    pthread_cond_init(&q->not_full, NULL);
    pthread_cond_init(&q->not_empty, NULL);
}

void queue_destroy(BoundedQueue *q) {
    pthread_mutex_destroy(&q->mutex);
    pthread_cond_destroy(&q->not_full);
    pthread_cond_destroy(&q->not_empty);
}

/* Producer calls this. Blocks if queue is full. */
void queue_push(BoundedQueue *q, int item) {
    pthread_mutex_lock(&q->mutex);
    while (q->count == CAPACITY && !q->shutdown) {
        pthread_cond_wait(&q->not_full, &q->mutex);
    }
    if (q->shutdown) {
        pthread_mutex_unlock(&q->mutex);
        return; /* don't push during/after shutdown */
    }
    q->buffer[q->tail] = item;
    q->tail = (q->tail + 1) % CAPACITY;
    q->count++;
    pthread_mutex_unlock(&q->mutex); /* unlock BEFORE signal -- avoids waking a thread that
                                     immediately blocks again on the mutex we still hold */
    pthread_cond_signal(&q->not_empty);
}

/* Consumer calls this. Blocks if queue is empty.
   Returns 1 with *out set on success, 0 if shut down and drained (no more items ever coming). */
int queue_pop(BoundedQueue *q, int *out) {
    pthread_mutex_lock(&q->mutex);
    while (q->count == 0 && !q->shutdown) {
        pthread_cond_wait(&q->not_empty, &q->mutex);
    }
    if (q->count == 0 && q->shutdown) {
        pthread_mutex_unlock(&q->mutex);
        return 0; /* nothing left, and no more coming */
    }
    *out = q->buffer[q->head];
    q->head = (q->head + 1) % CAPACITY;
    q->count--;
    pthread_mutex_unlock(&q->mutex);
    pthread_cond_signal(&q->not_full);
}

```

```

    }
    *out = q->buffer[q->head];
    q->head = (q->head + 1) % CAPACITY;
    q->count--;
    pthread_mutex_unlock(&q->mutex);
    pthread_cond_signal(&q->not_full);
    return 1;
}

void queue_shutdown(BoundedQueue *q) {
    pthread_mutex_lock(&q->mutex);
    q->shutdown = 1;
    pthread_mutex_unlock(&q->mutex);
    pthread_cond_broadcast(&q->not_empty);    /* wake ALL waiters so every one observes
        shutdown and exits */
    pthread_cond_broadcast(&q->not_full);
}

/* --- demo usage --- */
typedef struct { BoundedQueue *q; int id; int count; } ProducerArgs;

void *producer(void *arg) {
    ProducerArgs *pa = (ProducerArgs *)arg;
    for (int i = 0; i < pa->count; ++i) {
        queue_push(pa->q, pa->id * 1000 + i);
    }
    free(pa);
    return NULL;
}

void *consumer(void *arg) {
    BoundedQueue *q = (BoundedQueue *)arg;
    int item;
    while (queue_pop(q, &item)) {
        /* process item */
    }
    return NULL;
}

int main() {
    BoundedQueue q;
    queue_init(&q);

    pthread_t producers[3], consumers[4];
    for (int i = 0; i < 3; ++i) {
        ProducerArgs *pa = malloc(sizeof(ProducerArgs));
        pa->q = &q; pa->id = i; pa->count = 100;
        pthread_create(&producers[i], NULL, producer, pa);
    }
    for (int i = 0; i < 4; ++i) {
        pthread_create(&consumers[i], NULL, consumer, &q);
    }

    for (int i = 0; i < 3; ++i) pthread_join(producers[i], NULL);
    queue_shutdown(&q);
    for (int i = 0; i < 4; ++i) pthread_join(consumers[i], NULL);
}

```

```

    queue_destroy(&q);
    printf("All done\n");
    return 0;
}

```

Design points to narrate while coding (this is what separates pass/fail on this question):

1. **Circular buffer via head/tail/count modulo arithmetic.** Raw pthreads gives you no `std::queue` — a fixed-size ring buffer is the natural C data structure. Say out loud that you’re avoiding shifting elements on pop by wrapping indices with `% CAPACITY`.
2. **Two condition variables, not one.** `not_full` and `not_empty` are logically different waits. You *can* get away with one CV + `pthread_cond_broadcast` everywhere, but it’s wasteful (thundering herd). Mention the one-CV tradeoff if pushed.
3. **Unlock before signal.** Not required for correctness, but a real perf optimization — avoids an immediate context-switch-then-block for the woken thread.
4. **Shutdown protocol via flag + `pthread_cond_broadcast`** (not signal) — every blocked thread needs to wake and re-check, since broadcast is the only way to guarantee all of them observe the shutdown flag.
5. **while, not if,** per 1.4.
6. **`pthread_mutex_destroy/pthread_cond_destroy`** — pthreads objects need explicit teardown, unlike `std::mutex`’s destructor. Forgetting this is a real (if minor) resource leak; mention it even if you skip it in a rushed live-coding pass.

2.2 [Difficulty 2/5 - Easy-Medium] Follow-up Q: What if the queue should be unbounded (a linked list instead of a fixed array)?

Answer: Drop `not_full` entirely (producers never block) and switch buffer/head/tail/count for a linked list with a `push_back/pop_front`. Only `not_empty` remains for consumers.

```

typedef struct Node {
    int value;
    struct Node *next;
} Node;

typedef struct {
    Node *head, *tail;
    pthread_mutex_t mutex;
    pthread_cond_t not_empty;
    int shutdown;
} UnboundedQueue;

void uqueue_push(UnboundedQueue *q, int value) {
    Node *node = malloc(sizeof(Node));
    node->value = value;
    node->next = NULL;
    pthread_mutex_lock(&q->mutex);
    if (q->tail) { q->tail->next = node; q->tail = node; }
    else { q->head = q->tail = node; }
    pthread_mutex_unlock(&q->mutex);
    pthread_cond_signal(&q->not_empty);
}

/* pop: lock, wait while head==NULL && !shutdown, unlink q->head, unlock, free the old node,
   return value */

```

Mention the real-world tradeoff: unbounded means a fast producer can exhaust memory — this is exactly why bounded queues are what’s actually used in production thread pools and message queues.

2.3 [Difficulty 3/5 - Medium] Follow-up Q: How would you implement this with a counting semaphore instead of two CVs?

Answer: Use two semaphores — `empty_slots` initialized to `CAPACITY`, `filled_slots` initialized to 0 — plus a mutex that protects only the head/tail bookkeeping, not the blocking itself. This is the standard “bounded buffer problem” solution from the OS textbooks (Silberschatz), and pthreads-adjacent POSIX gives you `sem_t` directly (it’s declared in `<semaphore.h>`, a sibling POSIX header, not `pthread.h` itself, but it’s used constantly alongside pthreads and interviewers treat it as part of the same family).

```

#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

#define CAPACITY 10

typedef struct {
    int buffer[CAPACITY];
    int head, tail;
    sem_t empty_slots; /* counts free slots, starts at CAPACITY */
    sem_t filled_slots; /* counts filled slots, starts at 0 */
    pthread_mutex_t mutex; /* protects head/tail bookkeeping only */
} SemBoundedQueue;

void sem_queue_init(SemBoundedQueue *q) {
    q->head = q->tail = 0;
    sem_init(&q->empty_slots, 0, CAPACITY); /* 0 = shared between threads, not processes */
    sem_init(&q->filled_slots, 0, 0);
    pthread_mutex_init(&q->mutex, NULL);
}

void sem_queue_push(SemBoundedQueue *q, int item) {
    sem_wait(&q->empty_slots); /* blocks if no room */
    pthread_mutex_lock(&q->mutex);
    q->buffer[q->tail] = item;
    q->tail = (q->tail + 1) % CAPACITY;
    pthread_mutex_unlock(&q->mutex);
    sem_post(&q->filled_slots); /* signal one more item available */
}

int sem_queue_pop(SemBoundedQueue *q) {
    sem_wait(&q->filled_slots); /* blocks if nothing to take */
    pthread_mutex_lock(&q->mutex);
    int item = q->buffer[q->head];
    q->head = (q->head + 1) % CAPACITY;
    pthread_mutex_unlock(&q->mutex);
    sem_post(&q->empty_slots); /* signal one more slot free */
    return item;
}

void sem_queue_destroy(SemBoundedQueue *q) {
    sem_destroy(&q->empty_slots);
    sem_destroy(&q->filled_slots);
    pthread_mutex_destroy(&q->mutex);
}

```

Why mention this even if not asked: semaphores model “N permits available” more directly than CV+predicate. If the interviewer is from an OS-heavy background (kernel, embedded), they may specifically want to see this version. Note the mutex-held region here is extremely short (just index bookkeeping) since the semaphores do all the blocking — a nice performance observation to make out loud.

Second argument to sem_init explained: sem_init(sem, pshared, value) — pshared = 0 means the semaphore is shared between threads of the same process (what we want here); pshared = 1 means it lives in shared memory and can be used across process boundaries. Interviewers occasionally ask you to explain this second argument specifically since it’s a common point of confusion.

2.4 [Difficulty 2/5 - Easy-Medium] Q: Multiple producers AND multiple consumers on the same queue — does your solution from 2.1 still work?

Answer: Yes. The mutex serializes all access regardless of how many producers/consumers exist. `pthread_cond_signal` on `not_empty` only wakes threads waiting on `not_empty` (consumers) — producers wait on the *separate* `not_full` CV, so there's no cross-wakeup confusion. `pthread_cond_broadcast` during shutdown ensures no thread is left stranded on either CV.

Common follow-up: “Is `pthread_cond_signal` guaranteed to wake exactly one thread, and can you control *which* one?”
Answer: it wakes at least one, but POSIX does not specify a fairness/ordering guarantee (it's not necessarily FIFO by wait order) — most implementations (glibc's NPTL on Linux) do wake in roughly FIFO order in practice, but the standard doesn't require it, so don't design correctness around a specific wake order.

TIER 3 — PRIORITY-QUEUE-BASED TIMERS

3.1 [Difficulty 4/5 - Hard] Q: Design a timer service. `schedule(delay_ms, callback)` should run `callback` after `delay_ms` milliseconds. Support many pending timers efficiently.

What they're testing: can you combine a min-heap (soonest-expiring timer at top) with a single background thread + CV, instead of one thread per timer. In pure C there's no `std::priority_queue`, so you hand-roll a binary min-heap over an array, and no `std::chrono`, so you use POSIX `clock_gettime/struct timespec` and `pthread_cond_timedwait` (the pthreads equivalent of `wait_until`). No `std::function` either — callbacks are plain C function pointers with a `void*` argument, exactly like `pthread_create`'s start routine.

Key insight to say out loud immediately: “I'll use a min-heap ordered by absolute expiration time, with a single dedicated thread that always sleeps via `pthread_cond_timedwait` until the *next* soonest timer, rather than spawning a thread per timer or polling in a busy loop.”

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_TIMERS 1024

typedef void (*TimerCallback)(void *user_data);

typedef struct {
    struct timespec when; /* absolute expiration time */
    TimerCallback callback;
    void *user_data;
} TimerEntry;

typedef struct {
    TimerEntry heap[MAX_TIMERS];
    int size;
    pthread_mutex_t mutex;
    pthread_cond_t cv;
    pthread_t worker;
    int stop;
} TimerService;

/* --- timespec helpers --- */
static int timespec_cmp(const struct timespec *a, const struct timespec *b) {
    if (a->tv_sec != b->tv_sec) return a->tv_sec < b->tv_sec ? -1 : 1;
    if (a->tv_nsec != b->tv_nsec) return a->tv_nsec < b->tv_nsec ? -1 : 1;
    return 0;
}
```

```

static struct timespec timespec_add_ms(struct timespec ts, int ms) {
    ts.tv_sec += ms / 1000;
    ts.tv_nsec += (long)(ms % 1000) * 1000000L;
    if (ts.tv_nsec >= 1000000000L) { ts.tv_nsec -= 1000000000L; ts.tv_sec += 1; }
    return ts;
}

/* --- binary min-heap over heap[], ordered by .when (soonest first) --- */
static void heap_swap(TimerEntry *a, TimerEntry *b) { TimerEntry t = *a; *a = *b; *b = t; }

static void heap_push(TimerService *ts, TimerEntry entry) {
    int i = ts->size++;
    ts->heap[i] = entry;
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (timespec_cmp(&ts->heap[i].when, &ts->heap[parent].when) < 0) {
            heap_swap(&ts->heap[i], &ts->heap[parent]);
            i = parent;
        } else break;
    }
}

static TimerEntry heap_pop(TimerService *ts) {
    TimerEntry top = ts->heap[0];
    ts->heap[0] = ts->heap[--ts->size];
    int i = 0;
    for (;;) {
        int left = 2 * i + 1, right = 2 * i + 2, smallest = i;
        if (left < ts->size && timespec_cmp(&ts->heap[left].when, &ts->heap[smallest].when) < 0) smallest = left;
        if (right < ts->size && timespec_cmp(&ts->heap[right].when, &ts->heap[smallest].when) < 0) smallest = right;
        if (smallest == i) break;
        heap_swap(&ts->heap[i], &ts->heap[smallest]);
        i = smallest;
    }
    return top;
}

/* --- worker thread --- */
static void *timer_worker(void *arg) {
    TimerService *ts = (TimerService *)arg;
    pthread_mutex_lock(&ts->mutex);
    while (!ts->stop) {
        if (ts->size == 0) {
            pthread_cond_wait(&ts->cv, &ts->mutex); /* nothing scheduled -- sleep until
                                                    schedule() wakes us */
            continue;
        }
        struct timespec next_deadline = ts->heap[0].when;
        int rc = pthread_cond_timedwait(&ts->cv, &ts->mutex, &next_deadline);
        if (rc == ETIMEDOUT) {
            struct timespec now;
            clock_gettime(CLOCK_REALTIME, &now);
            if (ts->size > 0 && timespec_cmp(&ts->heap[0].when, &now) <= 0) {
                TimerEntry entry = heap_pop(ts);
                pthread_mutex_unlock(&ts->mutex); /* run callback WITHOUT holding the lock --
                                                    callback might call schedule() again -> would deadlock */
                entry.callback(entry.user_data);
            }
        }
    }
}

```

```

        pthread_mutex_lock(&ts->mutex);
    }
}
/* rc == 0 (real signal) or spurious wakeup: loop back, re-check ts->stop and
   re-read heap[0] */
}
pthread_mutex_unlock(&ts->mutex);
return NULL;
}

void timer_service_init(TimerService *ts) {
    ts->size = 0;
    ts->stop = 0;
    pthread_mutex_init(&ts->mutex, NULL);
    pthread_cond_init(&ts->cv, NULL);
    pthread_create(&ts->worker, NULL, timer_worker, ts);
}

void timer_service_schedule(TimerService *ts, int delay_ms, TimerCallback cb, void *user_data) {
    struct timespec now;
    clock_gettime(CLOCK_REALTIME, &now);
    TimerEntry entry;
    entry.when = timespec_add_ms(now, delay_ms);
    entry.callback = cb;
    entry.user_data = user_data;

    pthread_mutex_lock(&ts->mutex);
    heap_push(ts, entry);
    pthread_mutex_unlock(&ts->mutex);
    pthread_cond_signal(&ts->cv); /* a new timer might now be the soonest -- wake worker
                                   to recompute its wait */
}

void timer_service_shutdown(TimerService *ts) {
    pthread_mutex_lock(&ts->mutex);
    ts->stop = 1;
    pthread_mutex_unlock(&ts->mutex);
    pthread_cond_signal(&ts->cv);
    pthread_join(ts->worker, NULL);
    pthread_mutex_destroy(&ts->mutex);
    pthread_cond_destroy(&ts->cv);
}

/* --- demo --- */
void print_message(void *user_data) {
    printf("fired: %s\n", (char *)user_data);
}

int main() {
    TimerService ts;
    timer_service_init(&ts);
    timer_service_schedule(&ts, 500, print_message, "500ms");
    timer_service_schedule(&ts, 100, print_message, "100ms");
    timer_service_schedule(&ts, 1000, print_message, "1000ms");
    sleep(2); /* let all timers fire before shutdown -- see note below on graceful drain */
    timer_service_shutdown(&ts);
    return 0;
}

```

```
}
```

pthread_cond_timedwait vs std::condition_variable::wait_until, explain the difference explicitly: pthread_cond_timedwait takes an **absolute** struct timespec deadline (wall-clock time via CLOCK_REALTIME by default, or CLOCK_MONOTONIC if you set that clock on the pthread_condattr_t at creation — mention CLOCK_MONOTONIC is generally preferred for timers since it's immune to system clock adjustments/NTP jumps, unlike CLOCK_REALTIME), not a relative duration — this is the single most common bug spot: forgetting to add the delay to “now” and instead passing the delay itself as if it were an absolute time (which would wait until 1970 plus a few hundred milliseconds, i.e. return instantly, forever). It returns 0 on a real wakeup and ETIMEDOUT specifically when the deadline passed — unlike std::cv_status, you check this via the raw integer return code.

Critical correctness point to narrate: pthread_cond_timedwait can return for three reasons: (a) ETIMEDOUT because the deadline passed, (b) 0 because pthread_cond_signal was called (new timer scheduled), (c) 0 due to a spurious wakeup. In all three cases the correct action is the same: **loop back, re-read heap[0], and recompute what to wait for**. This uniform re-check is why the code doesn't need to distinguish (b) from (c) at all — it only special-cases ETIMEDOUT, and even then it double-checks the deadline against clock_gettime before firing, since a stray ETIMEDOUT racing exactly at the boundary is possible.

Fixed-size array heap limitation, mention proactively: MAX_TIMERS is a compile-time cap here for simplicity — in production C you'd either heap-allocate and realloc the array when full (classic dynamic-array-growth doubling strategy) or use a fixed pool with a rejection/backpressure policy. Say this out loud; a fixed array is fine for interview purposes but you should flag it as a simplification.

3.2 [Difficulty 3/5 - Medium] Follow-up Q: How would you support recurring timers (e.g. “run every 200ms”)?

Answer: Add an interval_ms field (0 = one-shot) to TimerEntry. After popping and running a recurring entry, re-push it with when = now + interval_ms:

```
typedef struct {
    struct timespec when;
    TimerCallback callback;
    void *user_data;
    int interval_ms;    /* 0 = one-shot */
} TimerEntry;

/* in timer_worker, after popping `entry` and before/after running the callback: */
if (entry.interval_ms > 0) {
    struct timespec now;
    clock_gettime(CLOCK_REALTIME, &now);
    TimerEntry next = entry;
    next.when = timespec_add_ms(now, entry.interval_ms);
    pthread_mutex_lock(&ts->mutex);    /* re-lock if you'd unlocked to run the callback */
    heap_push(ts, next);
    pthread_mutex_unlock(&ts->mutex);
}
```

3.3 [Difficulty 4/5 - Hard] Follow-up Q: What if you need to cancel a scheduled timer? A hand-rolled array heap doesn't support removing an arbitrary element either.

Answer — same two standard approaches as the std:: edition, now in C:

1. **Lazy deletion (simplest, what real systems conceptually do):** give each timer entry a cancelled flag behind a pointer the caller keeps a handle to; cancel() just flips the flag; the worker checks it on pop and skips execution (and doesn't reschedule it if recurring) without trying to remove it from the heap early.

```
typedef struct {
    struct timespec when;
    TimerCallback callback;
    void *user_data;
    int interval_ms;
    volatile int *cancelled;    /* points at a flag the caller owns, or one we heap-allocate
                                and hand back */
}
```

```

} TimerEntry;

/* schedule() mallocs an int, sets it to 0, stores the pointer in the entry, and also returns
   that pointer to the caller as an opaque "TimerHandle". cancel(handle) does: *handle = 1;
   the worker checks `if (*entry.cancelled) { free_if_last_owner; continue; }` before calling back. */

```

Ownership/freing of the shared cancelled int needs a clear rule (e.g. reference-counted, or “the timer service always frees it after popping, and cancel() after that point is a documented no-op/race the caller must avoid” — pick one and state it explicitly, since this exact ambiguity is what interviewers probe next).

2. **Indexed heap:** maintain a side array/hash map from timer ID → current heap array index, updated on every swap inside heap_push/heap_pop, so you can implement heap_remove_at(index) in O(log n) instead of waiting for lazy skip. Mention it exists; only implement it if explicitly asked, since it roughly doubles the code (every swap must also update the index map).

3.4 [Difficulty 2/5 - Easy-Medium] Q: Why a single timer thread instead of one thread per scheduled timer?

Answer: A thread per timer doesn’t scale — pthreads stacks default to a substantial reservation (pthread_attr_getstacksize typically defaults to 8MB on Linux unless you shrink it with pthread_attr_setstacksize), so thousands of pending timers means thousands of mostly-idle threads and heavy kernel scheduling overhead, plus cancelling a sleeping thread cleanly is awkward (pthreads has pthread_cancel, but it’s widely considered dangerous to use casually since it can interrupt a thread mid-critical-section unless cancellation points/cleanup handlers are carefully managed — worth naming as a reason to avoid the naive “thread per timer + pthread_cancel to cancel” design entirely). One thread + a heap + pthread_cond_timedwait is the standard pattern. At very high timer counts (millions), mention **timer wheels** by name as the next-level answer — bucketing timers into a circular array of time slots for O(1) insert/expire at the cost of coarser granularity, which is roughly how the Linux kernel’s own timer subsystem and things like Netty’s HashedWheelTimer work.

TIER 4 — THREAD POOLS

4.1 [Difficulty 3/5 - Medium] Q: Implement a thread pool. submit(task) queues work; a fixed set of worker threads execute it; support graceful shutdown.

What they’re testing: combines everything above — bounded queue, CV signaling, clean lifecycle — using pure pthreads. Since pthread_create’s start routine must be void*(*)(void*), and we want an OOP-style pool with member state, the cleanest way to bridge that gap is a C++ **class with a static trampoline function** that casts void* back to this and calls a real member function. This is the standard, idiomatic way experienced C++/pthreads engineers wire up object state to pthreads’ C callback signature — mention this pattern by name (“static trampoline”) if asked why you didn’t just make worker_loop itself the thread entry point.

```

#include <pthread.h>
#include <queue>
#include <vector>
#include <cstdio>

/* Plain struct instead of std::function, since std::function pulls in
   type erasure machinery we don't need here -- a function pointer + void* arg
   is the pthreads-native way to represent "a task", and mirrors pthread_create itself. */
struct Task {
    void (*fn)(void *);
    void *arg;
};

class ThreadPool {
public:
    explicit ThreadPool(int num_threads) : stop_(false) {
        pthread_mutex_init(&mutex_, NULL);
        pthread_cond_init(&cv_, NULL);
    }

```

```

workers_.resize(num_threads);
for (int i = 0; i < num_threads; ++i) {
    /* &ThreadPool::worker_trampoline is a static member function --
       it has the exact void*(*)(void*) signature pthread_create requires.
       We pass `this` as the argument so the trampoline can get back to real object state. */
    pthread_create(&workers_[i], NULL, ThreadPool::worker_trampoline, this);
}

~ThreadPool() {
    shutdown();
    pthread_mutex_destroy(&mutex_);
    pthread_cond_destroy(&cv_);
}

void submit(void (*fn)(void *), void *arg) {
    pthread_mutex_lock(&mutex_);
    if (stop_) { pthread_mutex_unlock(&mutex_); return; } /* reject new work after
       shutdown started */
    Task t; t.fn = fn; t.arg = arg;
    tasks_.push(t);
    pthread_mutex_unlock(&mutex_);
    pthread_cond_signal(&cv_); /* exactly one idle worker needs to wake for one new
       task */
}

void shutdown() {
    pthread_mutex_lock(&mutex_);
    if (stop_) { pthread_mutex_unlock(&mutex_); return; } /* idempotent */
    stop_ = true;
    pthread_mutex_unlock(&mutex_);
    pthread_cond_broadcast(&cv_); /* wake every worker so each can
       observe stop_ and drain the queue */
    for (size_t i = 0; i < workers_.size(); ++i) {
        pthread_join(workers_[i], NULL);
    }
}

private:
static void *worker_trampoline(void *arg) {
    ThreadPool *pool = static_cast<ThreadPool *>(arg);
    pool->worker_loop();
    return NULL;
}

void worker_loop() {
    for (;;) {
        Task task;
        pthread_mutex_lock(&mutex_);
        while (tasks_.empty() && !stop_) {
            pthread_cond_wait(&cv_, &mutex_);
        }
        /* Exit condition: stopping AND no more queued work (drain first, then exit) */
        if (stop_ && tasks_.empty()) {
            pthread_mutex_unlock(&mutex_);
            return;
        }
    }
}

```

```

        task = tasks_.front();
        tasks_.pop();
        pthread_mutex_unlock(&mutex_);    /* unlock before running task -- never hold the pool's
                                           lock while executing user code */

        task.fn(task.arg);
    }
}

std::vector<pthread_t> workers_;
std::queue<Task> tasks_;
pthread_mutex_t mutex_;
pthread_cond_t cv_;
bool stop_;
};

/* --- demo --- */
void print_task(void *arg) {
    int id = *static_cast<int *>(arg);
    printf("task %d running on thread %lu\n", id, (unsigned long)pthread_self());
    delete static_cast<int *>(arg);
}

int main() {
    ThreadPool pool(4);
    for (int i = 0; i < 20; ++i) {
        int *arg = new int(i);
        pool.submit(print_task, arg);
    }
    pool.shutdown();
    return 0;
}

```

Narrate while coding: 1. **Static trampoline pattern.** `pthread_create` needs a bare function pointer with signature `void*(*)(void*)` — it cannot take a non-static member function (those implicitly take a hidden `this` and don't match that signature). The fix: a static member function (`worker_trampoline`) that matches the required signature, given this as its `void*` argument, which immediately casts back and calls the real instance method. This is *the* idiomatic bridge between pthreads' C API and C++ object state — expect to be asked to explain or write this from scratch. 2. **Task as a function-pointer-plus-void* struct,** not `std::function`, keeps this fully in the “pthreads-native” style — it's the same shape as `pthread_create`'s own arguments, which is a nice thing to point out as intentional symmetry. 3. **Exit condition is `stop_ && tasks_.empty()`, not just `stop_`** — the #1 bug candidates introduce. Exiting as soon as `stop_` is true drops queued work on the floor. 4. **Never hold `mutex_` while running `task.fn(task.arg)`** — arbitrary user code could take a long time or itself call `pool.submit()` again (re-entrant submission), which would deadlock if the lock were still held. 5. **`pthread_cond_signal` on submit, `pthread_cond_broadcast` on shutdown** — same logic as the bounded queue in Tier 2: one new task needs to wake exactly one idle worker; a shutdown needs every worker to wake and re-check.

4.2 [Difficulty 4/5 - Medium-Hard] Follow-up Q: How do you get a result back from a task, since raw pthreads has no `std::future`?

Answer: Build a minimal result-holder yourself: a small struct with a `pthread_mutex_t` + `pthread_cond_t` + a “ready” flag + a result slot — this is, at its core, exactly what `std::future`/`std::promise` are implemented on top of internally, so writing one is a very instructive exercise and a strong signal if you can do it live.

```

#include <pthread.h>
#include <cstdio>

template <typename T>
class SimpleFuture {
public:

```

```

SimpleFuture() : ready_(false) {
    pthread_mutex_init(&mutex_, NULL);
    pthread_cond_init(&cv_, NULL);
}
~SimpleFuture() {
    pthread_mutex_destroy(&mutex_);
    pthread_cond_destroy(&cv_);
}

/* Called by the worker thread once the task completes. */
void set_value(const T &value) {
    pthread_mutex_lock(&mutex_);
    value_ = value;
    ready_ = true;
    pthread_mutex_unlock(&mutex_);
    pthread_cond_broadcast(&cv_); /* broadcast: possibly multiple threads calling
                                   get() */
}

/* Called by whoever wants the result -- blocks until set_value() runs. */
T get() {
    pthread_mutex_lock(&mutex_);
    while (!ready_) {
        pthread_cond_wait(&cv_, &mutex_);
    }
    T result = value_;
    pthread_mutex_unlock(&mutex_);
    return result;
}

private:
    pthread_mutex_t mutex_;
    pthread_cond_t cv_;
    bool ready_;
    T value_;
};

/* Usage: pass a SimpleFuture<int>* through a task argument struct, have the
wrapped function call future->set_value(result) as its last step, and
whichever thread needs the answer calls future->get() (blocks until ready). */
struct ComputeArgs {
    int input;
    SimpleFuture<int> *future;
};

void *compute_task(void *arg) {
    ComputeArgs *args = static_cast<ComputeArgs *>(arg);
    int result = args->input * args->input; /* pretend this is expensive work */
    args->future->set_value(result);
    delete args;
    return NULL;
}

int main() {
    SimpleFuture<int> future;
    ComputeArgs *args = new ComputeArgs{7, &future};
    pthread_t t;
    pthread_create(&t, NULL, compute_task, args);

```

```

int result = future.get(); /* blocks until compute_task calls set_value */
printf("result: %d\n", result);
pthread_join(t, NULL);
return 0;
}

```

Why this matters to say out loud: this is literally the producer-consumer pattern from Tier 2 again, specialized to exactly one item ever being produced (the result) and exactly one flag transition (`ready_ false → true`) — recognizing and saying “a future is just a one-shot, single-item producer-consumer channel” is a strong senior-level signal, and shows you understand *why* futures are built the way they are rather than just knowing the API.

4.3 [Difficulty 3/5 - Medium] Follow-up Q: How would you support task priorities (some tasks should run before others)?

Answer: Swap `std::queue<Task>` for the same hand-rolled binary min-heap from Tier 3 (or a `std::priority_queue<Task>` if you’re allowed a small amount of STL container usage alongside pure-threads synchronization — clarify with the interviewer whether “threads-only” means the synchronization primitives specifically, or the entire program including containers; most interviewers mean the former). Everything else — mutex, CV, worker loop, trampoline — is unchanged. Point out explicitly: **the thread pool and the Tier 3 timer service share the exact same skeleton** (mutex + CV + heap/queue + worker loop), just with a different comparator and a different wake condition (queue non-empty, vs. soonest deadline). Saying this pattern-reuse out loud is a strong signal.

4.4 [Difficulty 2/5 - Easy-Medium] Follow-up Q: What’s the right number of threads for the pool?

Answer, the standard framework: - **CPU-bound work:** `sysconf(_SC_NPROCESSORS_ONLN)` (POSIX’s way to get logical core count — the pthreads-world equivalent of `std::thread::hardware_concurrency()`) is the right ballpark. More adds context-switch overhead with no throughput gain. - **I/O-bound work:** more threads than cores, since most are sleeping on I/O at any moment (classic formula: `threads = cores * (1 + wait_time/compute_time)`), or better, use an async I/O model (`epoll/io_uring`) instead of blocking threads for very high I/O concurrency.

```

#include <unistd.h>
long num_cores = sysconf(_SC_NPROCESSORS_ONLN);

```

4.5 [Difficulty 3/5 - Medium] Q: What happens if a task function crashes or is buggy inside worker_loop? How do you make the pool robust?

Answer: Unlike C++ exceptions (which at least unwind predictably and can be caught), a raw C function pointer that segfaults or calls `abort()` will take down the entire process — there’s no pthreads-level try/catch equivalent for hardware faults. What you *can* do: - If the task might throw a C++ exception (since `Task::fn` could itself be C++ code even though its signature is C-style), wrap the call in `try/catch(...)` inside `worker_loop` so one bad task doesn’t call `std::terminate()` and kill every other worker’s in-flight work too. - For true crash isolation (a task might segfault), the only real fix is process-level isolation — run untrusted/risky tasks in a forked child process rather than a thread, since threads share the address space and a crash in one takes all of them down together. Mention this as the honest answer rather than pretending pthreads can sandbox a crashing task; it fundamentally cannot, because threads share the same memory space by design.

TIER 5 — ADVANCED: ATOMICS, MEMORY ORDERING, LOCK-FREE

Expect these mainly in senior/staff systems interviews. Note up front: pure POSIX pthreads has no atomics API of its own — pthreads predates the C11/C++11 memory model. For lock-free code in a “pthreads-only” interview, you reach for compiler builtins: GCC/Clang’s `__atomic_*` builtins (which implement the C11/C++11 memory model exactly, just without requiring `<atomic>`) or the older, coarser `__sync_*` builtins. I’ll use `__atomic_*` throughout since it’s the modern, precise one and maps 1:1 to the `std::memory_order_*` concepts if you’ve seen those.

5.1 [Difficulty 2/5 - Easy-Medium] Q: How do you do a simple atomic counter without `std::atomic`, using only compiler builtins?

Answer:

```

#include <pthread.h>
#include <stdio.h>

long counter = 0;

void *increment_atomic(void *arg) {
    (void)arg;
    for (int i = 0; i < 100000; ++i) {
        __atomic_fetch_add(&counter, 1, __ATOMIC_RELAXED);
    }
    return NULL;
}

int main() {
    pthread_t threads[4];
    for (int i = 0; i < 4; ++i) pthread_create(&threads[i], NULL, increment_atomic, NULL);
    for (int i = 0; i < 4; ++i) pthread_join(threads[i], NULL);
    printf("Final: %ld\n", counter);    /* always 400000 */
    return 0;
}

```

`__atomic_fetch_add(ptr, value, memorder)` is a GCC/Clang builtin (part of their C11/C++11 memory-model support) that does an atomic read-modify-write regardless of language — works identically in plain C or C++. `__ATOMIC_RELAXED` here mirrors `std::memory_order_relaxed` exactly: only the increment itself is atomic, no ordering guarantee relative to other memory operations — correct here since we just need the final sum right, nothing else to synchronize.

5.2 [Difficulty 4/5 - Hard] Q: Explain acquire/release memory ordering and implement the classic “flag guards data” handoff using only compiler atomics.

Answer, plain English first: - **Release** (on a write): all ordinary memory writes that happened *before* this write, in program order, become visible to any thread that does an *acquire* read of the same variable and observes this value. - **Acquire** (on a read): if you read a value written with *release*, all writes that happened-before that release become visible to you after this read.

```

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <assert.h>

int data = 0;
int ready = 0;    /* plain int, made safe purely via __atomic_* calls on it -- no <atomic>
needed */

void *producer(void *arg) {
    (void)arg;
    data = 42;    /* (1) ordinary write */
    __atomic_store_n(&ready, 1, __ATOMIC_RELEASE);    /* (2) release -- publishes (1) to
acquirers */
    return NULL;
}

void *consumer(void *arg) {
    (void)arg;
    while (!__atomic_load_n(&ready, __ATOMIC_ACQUIRE)) {    /* (3) acquire -- spins until (2)
is visible */
        usleep(100);
    }
    assert(data == 42);    /* guaranteed to see (1), because (2) release-synchronizes-with
(3) */
}

```

```

    printf("consumer saw data = %d\n", data);
    return NULL;
}

int main() {
    pthread_t p, c;
    pthread_create(&c, NULL, consumer, NULL);
    pthread_create(&p, NULL, producer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}

```

Why `__ATOMIC_RELAXED` on both sides would be wrong, say this explicitly: with relaxed ordering, the compiler/CPU is free to reorder `data = 42` to appear *after* `ready's` store from the consumer's point of view. The consumer could observe `ready == 1` but `data == 0` — a genuine, standard-permitted reordering, not a hypothetical. Acquire/release is the minimum ordering strength that makes this handoff correct.

5.3 [Difficulty 5/5 - Very Hard] Q: Implement a lock-free stack (Treiber stack) using compare-and-swap, no mutex at all.

Answer: `__atomic_compare_exchange_n(ptr, expected, desired, weak, success_memorder, failure_memorder)` is the builtin CAS. It atomically does: “if `*ptr == *expected`, set `*ptr = desired` and return true; else write the current actual value into `*expected` and return false.”

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int value;
    struct Node *next;
} Node;

typedef struct {
    Node *head; /* accessed only via __atomic builtins -- never a plain read/write */
} LockFreeStack;

void stack_init(LockFreeStack *s) {
    s->head = NULL;
}

void stack_push(LockFreeStack *s, int value) {
    Node *new_node = malloc(sizeof(Node));
    new_node->value = value;
    new_node->next = __atomic_load_n(&s->head, __ATOMIC_RELAXED);
    /* Loop: try to swing head from what we last saw to our new node.
       If someone else pushed/popped in between, compare_exchange fails,
       writes the CURRENT head into new_node->next, and we retry. */
    while (!__atomic_compare_exchange_n(
        &s->head, &new_node->next, new_node,
        1 /* weak: allowed to fail spuriously, must be in a retry loop */,
        __ATOMIC_RELEASE /* success: publish new_node's contents to whoever acquires */,
        __ATOMIC_RELAXED /* failure: nothing to synchronize, just retry */)) {
        /* new_node->next was refreshed to the current head by the builtin itself on failure */
    }
}

/* Returns 1 and sets *out on success, 0 if the stack was empty. */

```

```

int stack_pop(LockFreeStack *s, int *out) {
    Node *old_head = __atomic_load_n(&s->head, __ATOMIC_ACQUIRE);
    while (old_head != NULL &&
           !__atomic_compare_exchange_n(
               &s->head, &old_head, old_head->next,
               1, __ATOMIC_ACQUIRE, __ATOMIC_RELAXED)) {
        /* old_head refreshed to current head on failure; loop retries */
    }
    if (old_head == NULL) return 0; /* stack was empty */
    *out = old_head->value;
    free(old_head); /* NOTE: unsafe in general under true concurrency -- see ABA
                     discussion below */
    return 1;
}

/* --- demo --- */
typedef struct { LockFreeStack *s; int start; int count; } PushArgs;
void *pusher(void *arg) {
    PushArgs *pa = (PushArgs *)arg;
    for (int i = 0; i < pa->count; ++i) stack_push(pa->s, pa->start + i);
    free(pa);
    return NULL;
}
int main() {
    LockFreeStack s;
    stack_init(&s);
    pthread_t threads[4];
    for (int i = 0; i < 4; ++i) {
        PushArgs *pa = malloc(sizeof(PushArgs));
        pa->s = &s; pa->start = i * 1000; pa->count = 1000;
        pthread_create(&threads[i], NULL, pusher, pa);
    }
    for (int i = 0; i < 4; ++i) pthread_join(threads[i], NULL);
    int count = 0, val;
    while (stack_pop(&s, &val)) count++;
    printf("popped: %d (expect 4000)\n", count);
    return 0;
}

```

weak vs strong CAS, explain the parameter: passing 1 (weak) means the builtin may fail spuriously even when `*ptr == *expected` — a real hardware consideration on architectures using load-linked/store-conditional instructions. Must be used inside a retry loop (which we have). Pass 0 (strong) only for a one-shot check outside a loop, where a spurious failure would be wrongly treated as “value genuinely changed.”

The ABA problem — bring this up unprompted, it’s the “do you actually get lock-free programming” signal: between reading `old_head` and the CAS, another thread could pop node A, push some other node, then push a *new* node that happens to be allocated at the exact same address as the freed A (a common allocator behavior — freed memory gets reused quickly). Your CAS sees `head == old_head`’s address and succeeds, thinking nothing changed — but the list structure underneath may be completely different now. The naive `free(old_head)` above is genuinely unsafe in a long-running, truly concurrent system for exactly this reason: another thread might still be mid-dereference on that memory in its own CAS retry loop when you free and potentially reuse it.

Real fixes, name-drop them, full implementation is out of interview scope: **hazard pointers** (each thread publishes which node(s) it’s currently touching; nobody frees a hazarded node), **epoch-based reclamation** (defer frees until all threads have passed a safe point), or a **tagged pointer / version counter** appended to the pointer (e.g. pack a monotonically incrementing counter into unused high bits, or use a struct { Node *ptr; uintptr_t tag; } with a double-width CAS via `__atomic_compare_exchange_n` on a 128-bit type where supported) so “the same address” becomes distinguishable from “truly unchanged.” In an interview, stating the hazard clearly and naming the standard fixes is worth more than trying to implement

hazard pointers live.

5.4 [Difficulty 3/5 - Medium] Q: What is false sharing? How do you detect and fix it in a pthreads/C program?

Answer: CPU caches operate on cache lines (typically 64 bytes), not individual variables. If two threads write to two different variables that happen to land in the same 64-byte cache line, every write by one thread invalidates the *entire line* in the other thread's cache, forcing a coherency-protocol re-fetch — even though the threads share no logical data. Pure performance bug, not correctness.

```
/* BAD: counter_a and counter_b likely share a cache line */
typedef struct {
    long counter_a;    /* written by thread A */
    long counter_b;    /* written by thread B */
} Counters;
```

```
/* FIX: pad each to its own cache line */
typedef struct {
    long value;
    char padding[64 - sizeof(long)];
} __attribute__((aligned(64))) PaddedCounter;
```

```
PaddedCounter counter_a, counter_b;    /* now guaranteed on separate cache lines */
```

`__attribute__((aligned(64)))` is the GCC/Clang way to force cache-line alignment in plain C (the C++17 `std::hardware_destructive_interference_size` doesn't apply here since we're avoiding post-C++11 standard library features; hardcoding 64, or defining `#define CACHE_LINE_SIZE 64` for a single point of change, is the pragmatic pthreads-era approach). **Detection:** `perf c2c` (Linux) specifically detects cache-line contention; unexpectedly high cache-misses/L1-dcache-load-misses in `perf stat` for code that “shouldn't” be touching much shared memory is the symptom to watch for.

5.5 [Difficulty 3/5 - Medium] Q: What's a reader-writer lock? Implement reader/writer access with `pthread_rwlock_t`.

Answer: Allows multiple concurrent readers OR one exclusive writer (never both).

```
#include <pthread.h>

pthread_rwlock_t rw_lock = PTHREAD_RWLOCK_INITIALIZER;
int shared_data = 0;

void *reader(void *arg) {
    (void)arg;
    pthread_rwlock_rdlock(&rw_lock);    /* multiple readers can hold this simultaneously */
    int local = shared_data;
    (void)local;
    pthread_rwlock_unlock(&rw_lock);
    return NULL;
}

void *writer(void *arg) {
    (void)arg;
    pthread_rwlock_wrlock(&rw_lock);    /* exclusive -- blocks all readers and writers */
    shared_data++;
    pthread_rwlock_unlock(&rw_lock);
    return NULL;
}
```

When it's worth it: only when reads vastly outnumber writes AND the critical section is non-trivial — cheap reads under heavy write contention can make a `pthread_rwlock_t` slower than a plain mutex, since the reader-count bookkeeping itself has overhead. Always caveat with “I'd benchmark against a plain mutex before committing to this.”

5.6 [Difficulty 3/5 - Medium] Q: `pthread_mutex_t` vs `atomics (__atomic_*)` — when would you pick one over the other?

Answer: Use atomics for single, simple values (counters, flags, single pointers) where the operation maps directly onto a hardware atomic instruction (load, store, fetch-add, CAS). Use a mutex whenever you need to atomically update *multiple* related pieces of state together, or the operation is too complex for a single CAS to express. Forcing multi-field invariants into lock-free atomics usually multiplies complexity (ABA, memory reclamation, torn reads across fields) far more than any perf win is worth — default to a mutex, and only go lock-free when profiling proves the mutex is the actual bottleneck. This shows engineering judgment rather than “lock-free is always better,” a common junior mistake interviewers specifically probe for.

TIER 6 — GOTCHAS, EDGE CASES, AND THINGS THAT TRIP PEOPLE UP LIVE

Quick-fire section — these are the “wait, actually...” follow-ups interviewers ask after your main solution looks right. All rated [Difficulty 2/5 - Easy-Medium] individually, but interviewers often chain several as rapid-fire follow-ups, which raises the effective difficulty of the exchange.

1. **Forgetting to check `pthread_create`’s return value.** It returns 0 on success, a positive errno-style code on failure (e.g. EAGAIN if you’ve hit a system thread limit) — it does NOT set the global `errno`. A silently-failed `pthread_create` means your “worker” never runs, which is a nasty bug to chase if you assumed success.
2. **Joining a thread twice, or never joining/detaching a joinable thread.** Calling `pthread_join` twice on the same thread is undefined behavior. Never joining or detaching a joinable thread leaks its resources (thread control block, sometimes stack) for the life of the process — every `pthread_create` needs an eventual `pthread_join` OR `pthread_detach`, never neither.
3. **Passing a pointer to a stack-local loop variable into `pthread_create`.** Covered in 1.2 — the classic “everyone’s thread sees the same final value of `i`” bug. Always heap-allocate a per-thread argument copy, or index into a stack array that’s guaranteed to outlive every thread.
4. **Calling `pthread_cond_signal/broadcast` without holding the associated mutex.** This is actually *legal* (POSIX permits it) and sometimes preferable for performance (signal after unlocking, as in Tier 2’s push/pop). The misconception is thinking you *must* hold the lock while signaling — you must hold it while *changing the predicate state*, not while calling `signal/broadcast` itself.
5. **Forgetting `pthread_cond_timedwait` takes an *absolute* deadline, not a relative duration.** Covered in 3.1 — the single most common bug in timer/timeout code. Passing “500” (meaning “500ms from now”) directly as the timespec instead of `now + 500ms` gives you a deadline in 1970, so the wait returns ETIMEDOUT instantly, every time.
6. **Using `PTHREAD_MUTEX_INITIALIZER` and then calling `pthread_mutex_destroy` on it as if `pthread_mutex_init` had been called, or vice versa.** Statically-initialized mutexes (= `PTHREAD_MUTEX_INITIALIZER`) still need eventual `pthread_mutex_destroy` in most implementations if you want to fully release any OS resources — don’t assume static initialization means no cleanup is ever needed. Conversely, never call `pthread_mutex_init` twice on the same mutex without destroying it in between.
7. **Recursively locking a non-recursive `pthread_mutex_t`.** If the same thread calls `pthread_mutex_lock` twice on the same non-recursive mutex without unlocking in between, that’s undefined behavior — typically it deadlocks the thread against itself. If you genuinely need reentrancy, initialize the mutex with `PTHREAD_MUTEX_RECURSIVE` via `pthread_mutexattr_settype`, though needing this is often itself a sign the design should be refactored — mention this judgment.
8. **`pthread_cancel` interrupting a thread mid-critical-section.** Asynchronous cancellation (the default in some configurations) can kill a thread while it holds a lock, permanently deadlocking every other thread waiting on that lock — there’s no equivalent of RAII unwinding to release it automatically. If cancellation is required, use deferred cancellation with explicit cancellation points and cleanup handlers (`pthread_cleanup_push/pop`), or better, avoid `pthread_cancel` entirely and use a cooperative “please stop” flag checked by the thread itself (the same `stop/shut down` boolean pattern used throughout this kit).
9. **Priority inversion.** A low-priority thread holds a lock a high-priority thread needs; a medium-priority thread preempts the low-priority holder, indirectly starving the high-priority thread. The pthreads-level fix is **priority inheritance**, configured via `pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT)` before `pthread_mutex_init` — this temporarily boosts the lock-holder’s priority to match the highest-priority waiter so a medium-priority thread can’t cut in line. Name-drop the Mars Pathfinder 1997 bug as the textbook real-world case.

10. **Busy-waiting with `usleep` in a loop instead of blocking on a condition variable.** If you catch yourself polling a flag with a `sleep` instead of using `pthread_cond_wait`, say out loud “I’d replace this polling with a condition variable to avoid wasted CPU and added latency,” even in throwaway demo code — interviewers notice when candidates silently accept polling as the final answer.

DIFFICULTY-ORDERED QUESTION INDEX (for structured practice)

If you want to drill strictly in increasing order of difficulty rather than tier-by-tier, work through this ordering:

#	Difficulty	Question	Section
1	1/5 Easy	Broken counter → mutex fix	1.1
2	1/5 Easy	Passing multiple arguments via a struct	1.2
3	1/5 Easy	Spurious wakeups, <code>while</code> vs <code>if</code>	1.4
4	2/5 Easy-Medium	Deadlock example + lock ordering / trylock-backoff	1.3
5	2/5 Easy-Medium	RAII-less lock cleanup discipline in C	1.5
6	2/5 Easy-Medium	Unbounded producer-consumer (linked list)	2.2
7	2/5 Easy-Medium	Multi-producer/multi-consumer correctness reasoning	2.4
8	2/5 Easy-Medium	Right number of threads for a pool	4.4
9	2/5 Easy-Medium	Why one timer thread, not one per timer	3.4
10	2/5 Easy-Medium	Atomic counter via <code>__atomic_fetch_add</code>	5.1
11	3/5 Medium	Bounded blocking queue (array-based, two CVs)	2.1
12	3/5 Medium	Semaphore-based bounded queue	2.3
13	3/5 Medium	Recurring timers	3.2
14	3/5 Medium	Basic thread pool with static trampoline	4.1
15	3/5 Medium	Task priorities in the pool	4.3
16	3/5 Medium	Crash/exception robustness in a worker loop	4.5
17	3/5 Medium	False sharing: cause, detection, fix	5.4
18	3/5 Medium	Reader-writer lock with <code>pthread_rwlock_t</code>	5.5
19	3/5 Medium	Mutex vs atomics — when to pick which	5.6
20	4/5 Medium-Hard	<code>SimpleFuture</code> built from scratch on pthreads	4.2
21	4/5 Hard	Priority-queue timer service (hand-rolled heap + <code>pthread_cond_timedwait</code>)	3.1
22	4/5 Hard	Cancelling a scheduled timer (lazy deletion / indexed heap)	3.3

#	Difficulty	Question	Section
23	4/5 Hard	Acquire/release memory ordering, flag-guards-data handoff	5.2
24	5/5 Very Hard	Lock-free Treiber stack with CAS + ABA discussion	5.3

How to use this table: start at #1 and don't let yourself skip ahead — each question either directly reuses a pattern from an earlier one, or explicitly contrasts with it (e.g. #11's two-CV queue is the direct ancestor of #14's thread pool and #21's timer service; #10's atomic counter is the on-ramp to #23's acquire/release and #24's CAS). If you can't do #11 cleanly from a blank file, #14 and #21 will be much harder than they need to be — don't skip the medium tier to get to the impressive-looking hard questions.

SUGGESTED STUDY / PRACTICE SCHEDULE

Given ~45-60 min interview slots, practice each of these **standalone, from a blank .c/.cpp file, with a timer**, until you can produce a compiling, correct version without looking at this doc. All commands assume `gcc file.c -pthread -o out` (or `g++ -std=c++11 file.cpp -pthread -o out` for the thread-pool/future sessions that use a class).

Session	Task	Target time
1	Items #1-3: broken counter → mutex → atomic, plus arg-passing and spurious wakeups, explained verbally	20 min
2	Item #4-5: deadlock repro + both fixes (lock ordering AND trylock-backoff)	20 min
3	Item #11: bounded blocking queue, array-based, two CVs, full shutdown protocol	35 min
4	Item #12: same queue, semaphore-based version	20 min
5	Item #21: timer service — hand-rolled min-heap + <code>pthread_cond_timedwait</code>	45 min
6	Item #22: add lazy-deletion cancellation to session 5's timer	15 min
7	Item #14: thread pool with static trampoline pattern, fire-and-forget tasks	30 min
8	Item #20: upgrade session 7's pool with a hand-rolled <code>SimpleFuture<T></code>	25 min
9	Item #23: acquire/release flag+data handoff with <code>__atomic_*</code> , explain why relaxed breaks it	15 min
10	Item #24: Treiber stack with <code>__atomic_compare_exchange_n</code> , explain ABA unprompted	30 min
11	Cold mock: pick #11, #14, or #21 at random, build it in one sitting with no notes, against the 45-60 min budget	60 min

Verbal drill (separate from coding — do this on a walk): for each glossary term, explain it out loud in one sentence, no notes. If you hesitate on any of: data race vs race condition, the static trampoline pattern, absolute vs relative deadlines in `pthread_cond_timedwait`, acquire/release, the ABA problem, or priority inversion — go back and re-read that section before your next mock.